lr_train

November 8, 2026

[1]:
```
import pandas as pd
```

[2]:
```
df = pd.read_csv("./data/train.csv")
```

[3]:
```
df.head()
```

[3]:

	id	text	label
0	1	Cipriani S.A. is an Italian hotel and leisure …	0
1	2	My Grandpa passed away 3 years ago from cancer…	0
2	3	"Petit Beurre -- Butter Cookies\n\n1 cup (2 st…	1
3	4	In an unprecedented move, a dedicated gamer ha…	1
4	5	Chile's copper industry has registered record …	0

[4]:
```
from sklearn.model_selection import train_test_split

X = df["text"]
y = df["label"]

X,y
```

[4]:
```
(0       Cipriani S.A. is an Italian hotel and leisure …
 1       My Grandpa passed away 3 years ago from cancer…
 2       "Petit Beurre -- Butter Cookies\n\n1 cup (2 st…
 3       In an unprecedented move, a dedicated gamer ha…
 4       Chile's copper industry has registered record …
                              …
 9995    Yes this review is 2 years after the film's re…
 9996    Event Horizon is a suspenseful and visually st…
 9997    The purpose of this study is to improve liver …
 9998    Donald Meltzer (1922-2004) was a Kleinian psyc…
 9999    Sure! Here's a simple recipe for Buffalo Chick…
 Name: text, Length: 10000, dtype: str,
 0       0
 1       0
 2       1
 3       1
 4       0
        ..
```

```
9995    0
9996    1
9997    1
9998    0
9999    1
Name: label, Length: 10000, dtype: int64)
```

[5]:
```
# Split data into train and test sets

RANDOM_SEED = 42
X_train, X_test, y_train, y_test = train_test_split(X,y, test_size=0.2,␣
 ↪random_state=RANDOM_SEED, stratify=y)

X_train, X_test, y_train, y_test
```

[5]:
```
(5748    Shortly after Dreamworks screwed up with the u…
3629    The Scottish politician George Galloway has ma…
7796    Barnett Frank (born March 31, 1940) is a forme…
5319    I'm looking for some advice on how to deal wit…
3198    Hoping for some advice here. I have a 3 year o…
                              …
9973    > Brain Tumors are one of the most common type…
3494    6 large day old cinnamon rolls\n3 cups vitamin…
7582    The Doves have become the first band in 10 yea…
7401    Fareed Ramadan (, November 4, 1961 - November …
265     We introduce the notions of a differentiable g…
Name: text, Length: 8000, dtype: str,
5864    Usually, Neural Networks models are trained wi…
1773    Ingredients\n\n4 sweet potatoes\n\n1/2 cup (10…
3335    Elissa Panush Benedek (born September 28, 1936…
9081    HMS Shannon was a 18-gun sloop of the Royal Na…
6969    I don't know - I'm way too old for a middle li…
                              …
1186    Ingredients:\n- 4 thin beef flank steaks (abou…
6696    3 -4 tablespoons finely chopped lemongrass (re…
9473    Chocolate Cake In The Instant Pot®\n\nIngredie…
9134    Sure, here's a possible body for a Wikipedia a…
4548    Ingredients\n\n- 1 pie crust\n- 1/2 cup white …
Name: text, Length: 2000, dtype: str,
5748    0
3629    1
7796    0
5319    1
3198    0
       ..
9973    1
3494    0
```

```
7582    1
7401    0
265     0
Name: label, Length: 8000, dtype: int64,
5864    0
1773    1
3335    0
9081    1
6969    0
       ..
1186    1
6696    0
9473    1
9134    1
4548    1
Name: label, Length: 2000, dtype: int64)
```

[6]:
```
X_train.shape, y_train.shape
```

[6]:
```
((8000,), (8000,))
```

[7]:
```
# try two approaches: tf-idf + logistic regression vs embedding + logistic␣
 ↪regression

# 1) TF-IDF + logistic regression

from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    ConfusionMatrixDisplay,
    f1_score,
    roc_auc_score,
)


pipeline = Pipeline([
    (
        "tfidf",
        TfidfVectorizer(
            ngram_range=(1, 2),
            max_features=50000,
            sublinear_tf=True,
        ),
    ),
```

```
    (
        "classifier",
        LogisticRegression(
            random_state=RANDOM_SEED,
            C=1.0,
            max_iter=1000,
        ),
    ),
])

pipeline.fit(X_train, y_train)

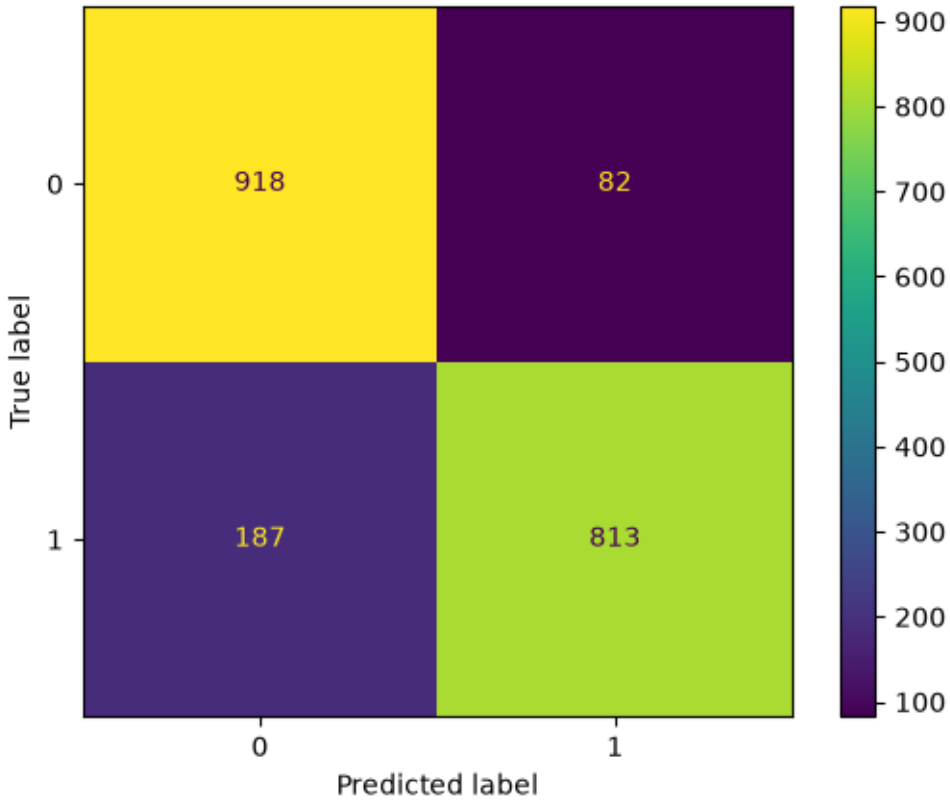
y_pred = pipeline.predict(X_test)
y_probability = pipeline.predict_proba(X_test)[:, 1]

cm = confusion_matrix(y_test, y_pred)
ConfusionMatrixDisplay(confusion_matrix=cm).plot()

accuracy = accuracy_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
roc_auc = roc_auc_score(y_test, y_probability)

print(f"Accuracy: {accuracy:.4f}")
print(f"F1 score: {f1:.4f}")
print(f"ROC-AUC: {roc_auc:.4f}")
```

```
Accuracy: 0.8655
F1 score: 0.8580
ROC-AUC: 0.9356
```



[8]:
```
import numpy as np
import ollama

from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    ConfusionMatrixDisplay,
    f1_score,
    roc_auc_score,
)


class OllamaEmbeddingTransformer(BaseEstimator, TransformerMixin):
    """Embed texts with a model served by the local Ollama daemon.

    Ollama answers a single /api/embed call with an error once the combined
    input gets too large (measured here: ~310 KB of text, i.e. roughly
    70k tokens, fails; 200 KB is fine). One batch of 32 short documents
```

```
    is ~40 KB, so there is plenty of headroom.
    Sending all 1600 training texts in one request therefore fails with
    "ResponseError: Post http://127.0.0.1:<port>/v1/tokenize ... connection
    reset by peer", so we embed in small batches instead.
    """

    def __init__(self, model="embeddinggemma-2:270m", batch_size=32):
        self.model = model
        self.batch_size = batch_size

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        texts = np.asarray(X, dtype=str).tolist()

        vectors = []
        for start in range(0, len(texts), self.batch_size):
            batch = texts[start : start + self.batch_size]
            response = ollama.embed(model=self.model, input=batch)
            vectors.extend(response["embeddings"])

        return np.asarray(vectors, dtype=np.float32)


pipeline = Pipeline([
    ("embedding", OllamaEmbeddingTransformer()),
    (
        "classifier",
        LogisticRegression(
            random_state=RANDOM_SEED,
            C=1.0,
            max_iter=1000,
        ),
    ),
])

pipeline.fit(X_train, y_train)

y_pred = pipeline.predict(X_test)
y_probability = pipeline.predict_proba(X_test)[:, 1]

cm = confusion_matrix(y_test, y_pred)
ConfusionMatrixDisplay(confusion_matrix=cm).plot()

accuracy = accuracy_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
```

```python
roc_auc = roc_auc_score(y_test, y_probability)

print(f"Accuracy: {accuracy:.4f}")
print(f"F1 score: {f1:.4f}")
print(f"ROC-AUC: {roc_auc:.4f}")
```

```
Accuracy: 0.8725
F1 score: 0.8666
ROC-AUC: 0.9263
```

